

Project 3 – Scrum Product Framing Document

Document Overview

- This document is a living artifact.
- It will evolve as the Product Backlog evolves.
- Sprint 0 is not for building features.
- Complete each section according to the instructions.

Section 1 – Product Goal (Long-Term Objective)

- Your product goal should be a clear, measurable statement describing the final state of the product.
- Requirements:
 - Must describe value.
 - Must be achievable by the end of the project timeline.
 - Must not describe tasks.
 - Must not describe activity.
 - Must describe outcome.
- Example format: By the end of this project, users will be able to _____ so that _____.
- Add your product goal in the space below.

By the end of the project, users will be motivated to track their biking journey so that they can make more efficient and environmentally friendly decisions when commuting.

Section 2 – Problem Statement

- What problem are we solving?
- Who experiences this problem?
- Why does it matter?
- What happens if this problem is not solved?

Write 1 – 2 strong paragraphs answering these questions in the space below.

Most cycling apps are built for athletes, not for everyday commuters. They focus on performance, health and personal records, completely ignoring another focus point: biking as a solution to urban pollution. There are no platforms that combine environmental impact tracking, community building, local partnerships, and practical commuter tools into one place. That gap means that established apps miss people who might want to bike to work or school instead of driving, having no real support system that helps them make decisions.

This matters because cities are running out of time to meet UN SDG goals around climate actions and sustainable transportation. When people lack personalized data showing them how much Co2 they're saving, how much money they're keeping in their pocket, or which trails and routes are available to them, the default choice stays the car.

Section 3 – Target User & Primary Persona

- Identify and define your primary user persona (the one driving prioritization decisions)
 - Age/Context:
 - Gen-Z age group: 14–30-year-olds. This allows us to personalize the website to match certain aesthetics, etc. The type of users that care about the environment and/or biking.
 - Goals
 - Allow users to comfortably start a biking journey that seamlessly replaces the car, prioritizing user needs over just environmental health.
 - Pain points
 - API integration and testing will be difficult. AI wrapper integration will be difficult as well as backend creation. Consistency in user experience will also be difficult.
 - Technical ability
 - Users need to understand how to navigate a website. As developers, it's our duty to make sure that we clearly section off areas, ensuring that viewers can easily navigate our product. Users must be comfortable with using a smart device and must have experience using "maps" of some sort.
 - a. Why they would use this product
 - i. There are no platforms that combine environmental impact tracking, community building, local partnerships, and practical commuter tools into one place. That gap means that established apps miss people who want to bike to work or school instead of driving, having no real support system that helps them make decisions. Users who are frustrated with existing products would use our website, especially if they care about the environment.
 - ii.
- Identify and define your secondary user persona (if relevant)
 - Age/Context
 - People who care about the environment but haven't factored in commuting as a possible focus point. People who are new to biking.
 - Goals
 - Allows users who don't have a lot of experience with biking, or who have recently started working on their CO2 output.
 - Pain points

- Finding trails for bikers of all abilities, especially since we as developers don't have much experience in this field.
 - We also need to figure out efficient paths for users to take and manage to map that to equivalent routes taken by car and public transport.
- Technical ability
 - Users need to understand how to navigate a website. As developers, it's our duty to make sure that we clearly section off areas, ensuring that viewers can easily navigate our product. Users must be comfortable with using a smart device and must have experience using “maps” of some sort.
- Why they would use this product
 - The secondary audience in this situation needs motivation to start their journey. Our audience is those who are either more environmentally focused (as opposed to biking), or those who have never used biking apps before. The cause of both scenarios above is usually a lack of exposure as well as not having access to a beginner friendly app that helps them integrate from a previous lifestyle. Getting access to such a product (ours) gives them the ability to begin combining energy conservation and transportation without much of a hassle on their end.
- If trade-offs occur, whose needs win?
 - If trade-offs occur, the needs of the customers/users must win because the end goal of Momentum is to encourage users to incorporate more sustainable practices into their daily lives. Sometimes sacrifices need to be made, and our team recognizes that in such situations, it is okay if the vision of the Product Manager is not fully met as long as users are satisfied with the end result.

Section 4 – Minimum Viable Product (MVP)

Define clearly:

1. What must exist for MVP?
2. What will NOT be included in MVP?
3. How will we know MVP is complete?
4. What can be demonstrated at MVP stage?

MVP must:

- Be demonstrable
- Be valuable
- Be constrained

Perfection is not MVP

The Minimum Viable Product (MVP) for Momentum will consist of a working web application with four main features. The user must be able to use the mapping interface to plan a bike route and log the activity in their “calendar”, calculate the carbon cost of different modes of transportation for their routes and view an estimate of the positive impact they make when they choose to bike, browse trail suggestions (must specify mileage, difficulty, elevation, and ratings from other users), and pin bike rides to a pin board to share with other app users.

Some features that will not be included in the MVP (that are intended to be incorporated later in the development process) are a community forum (for users of Momentum to share biking experiences), a bike marketplace, and an event management system (that displays nearby/recommended biking events based on user preferences).

As developers, we will know that the MVP is complete when the user is able to navigate our web application and successfully use all core features of the MVP. At the MVP stage, we should be able to demonstrate our working web application with all core features meeting our team’s definition of done.

Section 5 – Initial Product Backlog (Ordered)

Create an ordered list of 15-25 backlog items.

Each item must:

- Be written as a user story
- Be small enough to complete within a Sprint
- Be sized roughly (S/M/L)

Format: As a _____, I want _____ so that _____.

Highlight the top 5 highest priority items.

Important: This backlog will change.

1) As a biking enthusiast, I want to be able to see when there are local events or opportunities near me on the same platform as my other biking activity so that I can ensure that i don't miss opportunities and keep my schedule clean.

2) As an environmentalist who is new to biking, I want to be able to go on easy biking trips so that I can make biking a habit.

3) As a young student (Gen-Z), I want to be able to see when biking to a location will make sure I both attend class on time, as well as save gas money so that my parents don't get in trouble with my parents.

4) As a young student, I want to be able to save my memories of biking trips on the app so that I can share my memories with my friends.

5) As a biking enthusiast who doesn't know a lot about the environment, i want to be able track my CO2 savings so that I can better understand my part in the environment.

6) As a biking enthusiast who regularly visits the same bike shop, I want a community that can give me new suggestions that are environmentally vetted.

7) As a newcomer to the biking community, and as someone who cares for the environment, I want to be able to talk with more experienced bikers so that I can get a mentor of sorts.

8) As an avid biker, I want to be able to see the route I need to take to commute to work on the same platform so that I don't have to open multiple apps.

9) As an avid environmentalist I want to be able to see what biking shops near me are environmentally secure so that i can feel confident with my spendings.

Project 3 – Scrum Product Framing Document

10) As a frequent traveler, I want certain routes to be favorited and saved to my personal profile to minimize the effort required to commute.

11) As a biking enthusiast, I want personalized recommendations regarding the types of routes that the app thinks would align with me.

12) As a student who likes playing games, I want a platform that gamifies the experience of biking and saving energy.

13) As an avid bike collector, I want to be able to share with the world what sort of bike I have so that I can meet more people like me.

14) As someone who cares a lot about the environment, I want to be able to reach more people with my ideas because that is the only way that change is possible.

15) As someone who bikes in their free time with their family, I want to be able to utilize criteria like difficulty choosing routes that are optimal for me.

16. As a professional biker, I want to be able to be able to adjust my query bounds to align with routes that make sense to me.

Section 6 – Definition of Done (Team Agreement)

Work is only considered complete when:

- It works without errors.
- It can be demonstrated without explanation.
- It is merged into the main branch.
- It is peer reviewed
- It meets team quality standards.

Discuss, agree on, and explicitly write out your teams:

- Code standards (including style)
- Testing expectations
- Review requirements
- Merge policy
- Demo expectations.

All team members must sign this section.

Code Standards:

- Code Styling:
 - Allman Style Braces (all open braces start and close on a new line)
 - Indentation: tabs (or four spaces)
 - Line Length: 150 characters maximum
 - Naming Conventions:
 - Classes, Components, Methods: PascalCase
 - Variables, Functions: camelCase
 - Constants: SCREAMING_SNAKE_CASE
 - Always specify visibility (visibility will always be the first modifier)
 - ex: public abstract NOT abstract public
- Comments:
 - All files will have a header comment
 - All methods will be commented
 - No line of comment will exceed 100 characters
- File Styling:
 - All file names will be PascalCase
- GitHub:
 - Branch Naming: developer/feature
 - Use dashes (-) to separate spaces within feature

Project 3 – Scrum Product Framing Document

- Ex: Aarush/nav-bar
- Commit Messages
 - Short, yet descriptive
 - Should detail exactly what was worked on
 - If changes made, detail changes and why those changes were necessary
- Pull Requests:
 - Small and focused (no major changes at once to main branch)
 - Reviewed by at least one teammate
 - Message should specify what tester should look for

Testing Expectations:

- Define clear, measurable testing goals for testers
- Define test cases (edge cases, normal cases, errors)
- Create test data (to replicate what user might provide to software)
- Use multiple testing techniques
- Document testing (in testing README)

Review Requirements:

- Code reviewers must:
 - Ensure code meets code standards specificized above
 - Code is properly documented/commented
 - Code functionality is met (Definition of Done)
 - Test code according to Testing Expectations

Merge Policy:

- Merge small, focused commits
 - Large, unfocused commits tend to break existing code
- At least one other team member should test/review code before code is merged into main branch
- Each new feature added should be in its own individual branch
- Keep main updated to ensure that developer feature branches don't end up too far behind (leads to additional merge conflicts)
- CANNOT approve your own pull requests

Demo Expectations:

- Showcase completed work (Sprint Goal)
- Gather feedback from stakeholders

Project 3 – Scrum Product Framing Document

- Ensure work aligns with Product Manager’s vision
- Interactive (stakeholders should be allowed to interact with demo to gather valuable testing feedback)
- Update product backlog based on stakeholder feedback

Signatures:

Aarush Kapoor

Sriram Marasanapalle

Yohaán Ardeni

Sathkrith Segu

Section 7 – Development Tools & Technical Justification

Core Technology Choices - Complete the following

- Programming Language:
 - **Typescript (via React):** For frontend development, React is the industry standard and serves as a simple integration of HTML & CSS components. Yohaán and Sathkrith have done several React projects before, while Sriram & Aarush have worked on some JavaScript projects; however, the transition from JavaScript to TypeScript is not too difficult and will make development more effective and manageable. React handles dependencies simply and provides “components” that can be utilized several times to minimize structural redundancy throughout the site. Additionally, through the component structure, identification of vulnerabilities and errors will be much simpler and easily diagnosable. React does require Vite or a local development software to test, but that should be manageable on school laptops.
 - **Tailwind CSS:** As opposed to traditional CSS, Tailwind provides streamlined stylesheets that can be applied to modernize websites quickly and easily. All team members have familiarity with the language, and the utilization of the library should not be difficult. Since components are essentially just concise CSS components, they will increase readability significantly. Currently unsure about the limitations of Tailwind on school laptops, but it will serve as a dependency for our overall product.
 - **Python:** For backend features such as encryption and other API integrations, Python could be used to handle certain edge cases that React doesn’t have the bandwidth handle. All members have worked with Python during CSP and have done projects outside of school that require depth and familiarity with the language. As one of the most versatile and used languages in the world, we will likely run into no issues in terms of development as resources are nearly endless for issues we might run into. To run on school laptops, Python must be installed on the laptops, and since we are working on Windows, I would assume it is already installed.
- Framework/Engine (if applicable):
 - **React:** Provides an industry standard for UI development with options for client-side features that are easy to develop. More details about React are in the previous question under the Typescript language.
- Database (if applicable):
 - **Firestore or Supabase:** For backend, we want something that integrates seamlessly, and both Firestore and Supabase provide that. Firestore has an

extremely simple integration with React with minimal routing and excess code required. Supabase is a little more complicated but allows for tag-based querying and image capabilities. Both backend services are catered for React; however, we are unsure currently which one would be most applicable for us. This is something we will decide in one of our later Sprints after we finish the frontend and decide whether the tag-based image feature is worth the cost. Sathkrith has worked with Supabase before while the rest of the team has worked with Firebase. School laptops should have little to no obstacles to integrating with these backend services, so testing should not have any issues.

- **Hosting/Deployment Plan (if applicable):**
 - **Vercel:** Vercel is the perfect tool for React developments as their hosting service is free to use, lightweight, and handles environmental variables on their end. That means dependencies that might have to traditionally be handled within our code and might serve as vulnerabilities will be on the deployment side, making testing and addressing issues simpler. All of us have worked with Vercel before, and it requires little to no prior technical experience to work with. As for school limitations, it is unfortunately blocked by administration on the school provided laptops, but testing can still be done locally.
- **Additional Libraries/APIs (if applicable):**
 - **Mapping (Google Maps):** Our app is heavily based on the use of mapping; hence, we will depend on an API for generating maps, collecting time & distance data, and other visual components. We have used Google Maps API with barebones integration, but none of us have familiarity with the use of specific features that will be needed for this API. However, it is highly publicized and there is very likely a plethora of documentation available that would make the learning curve a lot easier to overcome. Unsure about school laptop limitations, but Google Maps itself does not have any admin blocking, so I would hope the API can be used too.

Why These Tools Are Appropriate – Write 1-2 well-developed paragraphs that address:

- How these tools support your product requirements
- Why they fit your team's skill level
- Whether they avoid creating a single point of failure
- Any classroom or technical constraints

Project 3 – Scrum Product Framing Document

Section 8 – Known Risks and Assumptions

Technical Risks (at least 2)

- Risk 1: **Vulnerability**
- Mitigation Plan: Our team intends to mitigate vulnerabilities during the development process of Momentum by employing a Pair programming strategy and conducting frequent peer reviews. Pair programming will improve code quality and reduce bugs/errors introduced into the program because having two people read through the code will ensure code is written cleanly and bugs are caught early. Conducting frequent peer reviews will help identify crucial design flaws that were missed by the developers and will offer a new perspective on the web application (one that doesn't make the same assumptions).
- Risk 2: **API Keys**
- Mitigation Plan: Our team intends to mitigate issues with API Keys during the development process of Momentum by ensuring secure storage/handling of the API Keys and monitoring access/usage. Some of the most common issues with API Keys are hardcoding API keys directly and storing them incorrectly, so ensuring safe handling practices will prevent accidentally exposing the API Keys and minimize the vulnerabilities created when an API Key becomes compromised.

Teamwork Risks (at least 1)

- Risk: **Procrastination**
- Mitigation Plan: Our team intends to mitigate procrastination during the development of Momentum by promoting accountability and having a structured development process. We will employ a pair programming strategy during class time to improve problem-solving and communication, and hold each other accountable to getting the Scrum Goals completed. We will also ensure that we have clear deadlines with small, manageable tasks to ensure that developers don't feel overwhelmed with the Sprint Goal.

Timeline Risks (at least 1)

- Risk: **Insufficient Testing Time**
- Mitigation Plan: One of the biggest reasons why developers skip testing during the development process is due to time mismanagement and not getting help when needed. Our team intends to mitigate insufficient testing time by ensuring that tasks have clear deadlines (with time for testing incorporated within those deadlines) so that

Project 3 – Scrum Product Framing Document

developers don't skip testing during the development process. We will also ensure that during our daily meetings we address any blockers that arise during the development process to ensure that developers get help if and when they need it.

Section 9 – Success Metrics

How will you measure if your product is successful?

Examples:

- Users can complete core flow
- All MVP features demonstrable
- No major runtime errors
- Performance benchmarks met

List 3-5 success metrics:

Success Metrics:

- Users can use the built-in mapping interface to plan a bike route
- Users can log a biking activity in their calendar (and see progress stats)
- Users can calculate the carbon cost of different modes of transportation when considering a route
- Users can use the web application without encountering any errors
- Users can interact with other community members (through pin boards, community forums, or marketplaces)